Appendix G
JIT Compilation versus AOT Compilation

Recall from Chapter 1 that the Java Virtual Machine provides a Just-In-Time (JIT) compiler, which translates Java bytecode into native machine code at runtime. Plus, the JIT compiler profiles the code as it is running to discover methods (hot spots) where additional optimizations can be performed. Performance improvements can be significant for methods that are executed repeatedly. Note that Java's JIT compiler is part of the Java Virtual Machine (JVM), not the actual Java compiler that translates Java source files into bytecode. The term JIT typically applies to a virtual machine.

But many compilers use a different compilation model called Ahead-Of-Time (AOT) compilation, whereby the source code is translated completely into a native executable format for a specific environment; e.g., the compiler (possibly together with a linker) creates a ".exe" file for a Windows/x86-64 computer. All translation and optimization are done at compile time, and the native code is run directly on the processor, not as part of a virtual machine. Practically all C and C++ compilers use AOT compilation.

So which compilation model is better, JIT or AOT? The answer is ... it depends. Native executables created by AOT compilers typically start faster and run faster initially, but JIT compilers can often provide better overall optimization based on runtime execution patterns. Many server-based applications (e.g., web applications) are designed to run for extended periods of time – months to possibly years. For these types of applications, JIT compilation is an excellent choice. But for many applications, startup time and initial performance are more important, and for these applications, AOT compilation is usually a better choice. Compilers tend to fall in the second category of applications.

The three major applications discussed in this book are the CPRL compiler, the CVM (virtual machine), and the assembler for CVM assembly language. Complete source code for the latter two applications is available on the book's GitHub repository. Through the development of the CPRL compiler, we assume that all three applications will run on the JVM. But none of these applications are designed to run for extended periods of time, and so it seems likely that they could benefit from AOT compilation. Let's find out.

Creating a Native Executable

There are several tools that can be used to convert JVM applications into native executables, including the Java Development Kit (JDK) packaging tool `jpackage` and the GraalVM tool `native-image`. Since the GraalVM `native-image` tool creates a pure, native binary without a JVM layer, we will use it to create native applications for the compiler, the assembler, and the CVM. The discussion that follows will create native applications for Windows/x86-64 computers, but it is relatively straightforward to create native applications for Apple or Linux computers using the GraalVM `native-image` tool.

We start by assuming a Windows/x86-64 environment and a project setup as described in Appendix A. We also assume that GraalVM is installed and that its `bin` subdirectory is on the execution search path (included in the `PATH` environment variable). Then we create a

subdirectory of the compiler project's `bin` directory (the one containing the scripts for compiling, assembling, and testing your CPRL test examples) named "`native`".

Using a text editor, add a script file named "`make_cprlc.cmd`" to directory `native` with the following contents.

```
@echo off

rem set config environment variables locally
setlocal
call ..\cprl_config.cmd

native-image -cp "%COMPILER_PROJECT_PATH%" -march=compatibility -O3 ^
   cprlc.Cprlc cprlc

rem restore settings
endlocal
```

The two key lines in the above listing are highlighted in bold. The "`-march`" and "`-O3`" options allow all features of the x86-64 with all optimizations for best performance. Note that the caret (^) character is the line continuation character for Windows command files. Typically you would just put everything on one line, but a single line was too long for the font and margins of this book.

For a bash/x86-64 environment, the script file "`make_cprlc`" (no "`.cmd`" suffix) would be written as follows, with the backslash (\) character serving as the line continuation character for bash scripts.

```
#!/bin/bash

# set config environment variables
source ../cprl_config

native-image -cp "$COMPILER_PROJECT_PATH" -march=compatibility -O3 \
   cprlc.Cprlc cprlc
```

In Windows, running `make_cprlc.cmd` from a command prompt will create the native executable file `cprlc.exe`. Follow a similar procedure to create native executables `assemble.exe` and `cprl.exe`.

Copy `setPath.cmd` and all `test...` files from the project's `bin` directory to the newly created `native` subdirectory. At this point the directory structure should look like the one shown on the next page (some files not shown).

Note that script files `assemble.cmd`, `cprl.cmd`, and `cprlc.cmd` in the parent directory have been replaced by native executables `assemble.exe`, `cprl.exe`, and `cprlc.exe` in the subdirectory. But since the other script files just call the executables by their base name (i.e., without the "`.cmd`" suffix), they will still run correctly from either directory. For example, when script file `testCorrect_all.cmd` calls `cprlc`, it will call either `cprlc.cmd` or `cprlc.exe` as appropriate.

Structure of the project `bin` directory with a subdirectory for native executables is as follows:

```
bin
├── assemble.cmd
├── cprl.cmd
├── cprlc.cmd
├── cprl_config.cmd
├── disassemble.cmd
├── setPath.cmd
├── testCorrect.cmd
├── testCorrect_all.cmd
├── testEverything.cmd
├── testIncorrect_all.cmd
└── native
    ├── assemble.exe
    ├── cprl.exe
    ├── cprlc.exe
    ├── make_assemble.cmd
    ├── make_cprl.cmd
    ├── make_cprlc.cmd
    ├── setPath.cmd
    ├── testCorrect.cmd
    ├── testCorrect_all.cmd
    ├── testEverything.cmd
    └── testIncorrect_all.cmd
```

Now let's see if the native executable files offer improved performance. Open two command prompts. In the first command prompt, change directory to the project's `bin` directory and run `setPath.cmd` in that directory to ensure that it is included in the execution search path. Then run `testEverything.cmd`, observing how long it takes to run. If the compiler project is correct and complete, then all tests should run with the expected results.

In the second command prompt, change directory to the `native` subdirectory and run `setPath.cmd` in that directory to ensure that it is included in the execution search path. Then run `testEverything.cmd`, observing how long it takes to run. You should observe that `testEverything.cmd` ran much faster in the `native` subdirectory than it did in the project's `bin` directory, but let's measure.

Measuring the Performance on Windows

Bash (and therefore Linux and Apple computers) has a `time` command that can be used to measure how long a given script takes to run. Windows PowerShell has a "cmdlet" (PowerShell term) called `Measure-Command` that does something similar, but the standard Windows Command Prompt doesn't seem to have a standard equivalent. There is an open-source utility called `ptime` that can be used for this purpose, but the web page for

`ptime` says that it was last tested under Windows XP in 2005 and may be obsolete. Let's write our own utility for this purpose in Java.

Omitting error checking for arguments, method `main()` for our timing program can be implemented similar to the following.

```
public static void main(String[] args)
    throws IOException, InterruptedException
  {
    System.out.println();
    System.out.println("===  " + args[0] + " ===");

    // for windows
    ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/c", args[0]);

    // for bash
    //ProcessBuilder pb = new ProcessBuilder("bash", "-c", args[0]);
    pb.redirectErrorStream(true);

    ...   // call System.nanoTime() to get start time

    Process p = pb.start();
    var reader =
          new BufferedReader(new InputStreamReader(p.getInputStream()));
    String line;
    while ((line = reader.readLine()) != null)
        System.out.println(line);

    ...   // call System.nanoTime() to get stop time
    ...   // compute and write out elapsed time
  }
```

Using the approach outlined in this appendix, we can create a native executable named `wtime` that can be used to measure the time needed to run `testEverything.cmd` as follows.

```
wtime testEverything
```

The execution of the script file `testEverything.cmd` can be timed using all three timing tools described previously in this appendix.

1. Windows PowerShell cmdlet `Measure-Command`
2. `ptime` (Yes, it still works on Windows 11!)
3. `wtime` (as described above)

All three timing tools give very similar results, with the native versions of the applications running much faster than the non-native (Java) versions.

For Windows PowerShell users, run these two commands separately in a PowerShell prompt.

```
$env:Path = "$PWD;" + $env:Path  # set path to include current dir
Measure-Command { testEverything | Out-Default }
```

But what happens if we run `testEverything` several times in a row? With Java, we would expect improved performance on the second run due to the "warm up" of the Java virtual machine, but there are additional factors at play for both the native and non-native versions of the applications. When we study topics such as computer organization and/or operating systems, we learn about cache memory and virtual memory via paging. With frequently-used blocks of memory already loaded into cache and virtual pages already moved into main memory, we would expect to get the best performance. In other words, when an application is run a second time or third time, we could expect it to run more quickly. Exact performance improvements can vary greatly across different computers based on relative processor speeds, operating systems, amount of cache memory, etc.

The following numbers were measured on my computer running Windows 11, The Java version was tested using OpenJDK version 26.0.1. The native (`.exe`) versions of the applications were created using GraalVM version 25.0.5. Before testing each version, I rebooted (restarted) the computer and then waited several minutes for the operating system to finish all initialization. I then ran `testEverything` multiple times consecutively, measuring the number of seconds for each run.

Run Times for the CVM

	Java Version	GraalVm (.exe) Version
1st Run	31.181 seconds	24.653 seconds
2nd Run	14.245 seconds	11.892 seconds
3rd Run	13.896 seconds	6.161 seconds

As might be expected, the second and third runs showed improved performance, even with the native (`.exe`) versions. Fourth and subsequent runs showed no meaningful improvement. As an aside, I should note that I have created versions of the compiler using C++, C#, and Kotlin, and although the actual numbers were different, all three of those compiler versions exhibited similar improvements over the first three runs.

So for Windows, the native versions of the compiler, assembler, and virtual machine proved to be faster than the JVM versions when run against the test examples. But what about Linux and other Unix-based operating systems such as macOS?

Measuring the Performance on Linux

Using a process similar to the one described above, native applications were also created for Linux. When script `testEverything` was run with these versions of the applications, the results were disappointing at first. But the initial tests were performed on Windows Subsystem for Linux 2 (WSL2), a lightweight virtual machine that runs on Windows.

For software developers that run Windows most of the time, WSL2 is a great tool that offers a nearly complete Linux environment in a lightweight virtual machine. Plus, it is possible to access the Linux files from Windows, and vice versa. For developers that use Windows but also want a native Linux experience without a virtual machine, it is possible to configure a single computer that dual boots either Windows or Linux; however, the two environments are totally separate.

So all testing was moved to a different computer that ran Linux directly (not as part of a virtual machine). Without going into details, here is a summary of the differences between the results measured on the first computer running Windows and the second computer running (Ubuntu) Linux. Note that only the Java versions of the applications were tested on Linux; i.e., no native executables were created.

- The first runs were only slightly faster on the second machine.
- The second and third runs on Linux showed no significant difference in run-time performance.

Measuring Application Size

What about application size? When GraalVM generates a native executable for an application, it needs to include a lot of the JVM environment as well as the application-specific code. But, surprisingly, the GraalVM-generated version of the CVM was approximately twice as large as the GraalVM-generated version of the compiler, even though the source code for the CVM was smaller. One would expect that a C version of the executable for the CVM would be much smaller. That was, indeed, the case for both the Windows and Linux versions as shown in the following table.

Sizes of Native Executables for the CVM

	GraalVM Version	C Version
Windows	16,992 KB	206 KB
Linux	17, 283 KB	33 KB

Plus, the C version ran slightly faster. Implementing the CVM in C is clearly a worthwhile exercise in performance improvement. However, it should be noted that other important factors such as development time and maintainability could be impacted negatively.

For reference, source code for the C implementation used in the above tests is provided in the `Handouts` directory of the GitHub repository for this book. (But don't peek until you have attempted exercise 13 in Appendix B yourself.)

Ahead-of-Time Caching

Every time a Java application starts, the Java Virtual Machine rebuilds the same classes from scratch, and the JIT compiler optimizes the same classes from scratch, which results in slow startup and warmup times. Introduced via JDK Enhancement Proposal (JEP) 483 in JDK 24, and enhanced in JDK 25 and 26, Ahead-of-Time (AOT) caching reduces

application startup and warmup times by loading an AOT cache directly from a disk file. The AOT cache is based on method execution profiles from a previous run of the application.

Using AOT caching is a two-step process. First a training run is used to build the cache. A training run is one performed deliberately, ahead of time, to let the JDK observe the running application and build the cache from what it observes. For optimal results, the training run should use workloads as close to production workloads as possible. Real users never see the training run. A production run, by contrast, is the deployment of the application using the finished cache to improve startup performance.

Here is a summary of the two steps.

1. Generate the AOT Cache

Run the application using the `-XX:AOTCacheOutput` flag.

```
java -XX:AOTCacheOutput=app.aot -cp app.jar com.example.App
```

2. Run the appication with the AOT Cache

Launch the application in production with the created cache file.

```
java -XX:AOTCache=app.aot -cp app.jar com.example.App
```

Caution: The generated `.aot` file is coupled exclusively to the exact application code, library versions, and JDK patch version. Any modification to the source code or platform requires a complete cache regeneration.

The impact of AOT caching was tested for the three main applications of the compiler project – the compiler, the assembler, and the CVM. The training runs used the CPRL test examples to create separate AOT caches for each application. Extensive testing using the `testEverything` script showed that, for these applications, using an AOT cache improved the overall performance of the applications by more than 19%. Testing of longer running server-based applications has shown more significant improvements, especially during startup.

If you are interested in AOT for Java, you should follow the progress of the OpenJDK project Leyden (see https://openjdk.org/projects/leyden/). In particular, the latest version of the Java Enhancement Proposal (JEP) entitled "Ahead-of-Time Code Compilation", currently in draft form, gives you a good idea of future directions of Java with respect to AOT.